

Assessment Tool-4

Compiler Design.

V. Peraradan.

192521404.

CSA1405.

Q1. Design an Unambiguous CFG and Shift-Reduce parser:

i) Unambiguous CFG:-

$\uparrow > * > -$
Associativity:-

$\uparrow$: Right associativity.

$*$: Left associative.

$-$: Left associative.

Grammar:

$E \rightarrow E - T \mid T$

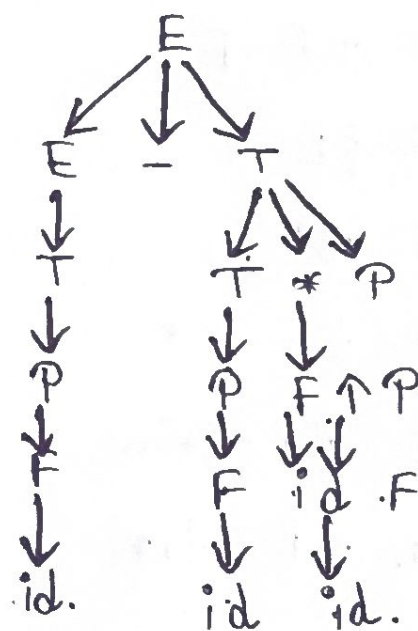
$T \rightarrow T * P \mid P$

$P \rightarrow F \uparrow P \mid F$

$F \rightarrow id \mid (E)$

This grammar removed ambiguity and correctly handled precedence.

(iv) Parse Tree:-



ii) Operator Precedence:-

Q.No	operator.	Precedence.	Associativity
1.	$\uparrow$	Highest.	Right.
2.	$*$	Medium.	Left
3.	$-$	Lowest	Left

iii) Shift-Reduce Parsing:-

Input:-

id - id * id $\uparrow$ id.

No.	Stack	Input.	Action.
1.	.\$	id - id * id ↑ id \$	Shift id.
2.	.\$id.	- id * id ↑ id \$	Reduce $F \rightarrow id$.
3.	.\$F	- id * id ↑ id \$	Reduce $P \rightarrow F$
4.	.\$P	- id * id ↑ id \$	Reduce $T \rightarrow P$.
5.	.\$T	- id * id ↑ id \$	Reduce $E \rightarrow T$.
6.	.\$E	- id * id ↑ id \$	Shift -
7.	.\$E-	id * id ↑ id \$	Shift id.
8.	.\$E-id.	* id ↑ id \$	Reduce $F \rightarrow id$.
9.	.\$E-F.	* id ↑ id \$	Reduce $P \rightarrow F$.
10.	.\$E-P	* id ↑ id \$	Reduce $T \rightarrow P$.
11.	.\$E-T	* id ↑ id \$	Shift *
12.	.\$E-T*	id ↑ id \$	Shift id.
13.	.\$E-T*id.	↑ id \$	Reduce $F \rightarrow id$.
14.	.\$E-T*F↑	↑ id \$.	Shift ↑
15.	.\$E-T*F↑	id \$	Shift id
16.	.\$E-T*F↑id.	.\$	Reduce $F \rightarrow id$.
17.	.\$E-T*P.	\$	Reduce $P \rightarrow F \uparrow P$
18.	.\$E-T	\$	Reduce $T \rightarrow T * P$
19.	.\$E.	\$	Reduce $E \rightarrow E - T$.
20.	Accept.		

Q2. LL(1) Parser :-
original Grammar:-

$D \rightarrow TL;$
 $L \rightarrow L, id / id$
 $T \rightarrow int / float.$

Left Recursion:-

$D \rightarrow TL;$

$L \rightarrow id L'$

$L' \rightarrow , id L' \mid \epsilon$

$T \rightarrow int \mid float.$

Follow sets:-

$Follow(D) = \{ \$ \}$

$Follow(T) = \{ id \}$

$Follow(L) = \{ ; \}$

$Follow(L') = \{ ; \}$

ii) FIRST sets:-

$FIRST(D) = \{ int, float \}$

$FIRST(T) = \{ int, float \}$

$FIRST(L) = \{ id \}$

$FIRST(L') = \{ ', ', \epsilon \}$

iii) LL(1) Parsing Table.

Non-Terminal	int	float	id	,	;	\$
D	$D \rightarrow TL;$	$D \rightarrow TL;$	-	-	-	-
T	$T \rightarrow int$	$T \rightarrow float$	$L \rightarrow id L'$	-	-	-
L	-	-	-	$L' \rightarrow , id L'$	$L' \rightarrow \epsilon$	-
L'	-	-	-	-	-	-

iv) Parse Input:-

int id, id;

Stack:-

\$D.

\$;LT

\$;Lint

match int.

match id.

Expand L'

match,

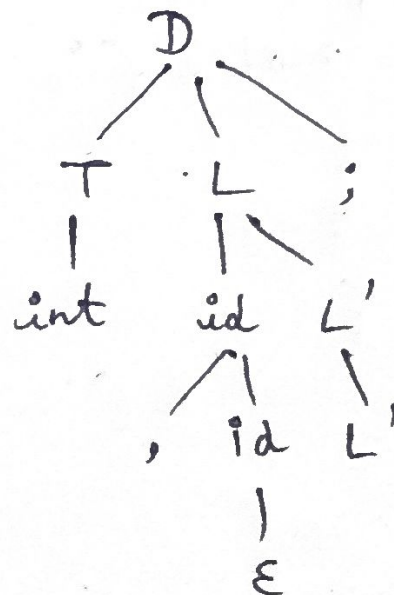
match id.

$L' \rightarrow \epsilon$

match;

Accept.

Parse Tree:-



Q3. Recursive Descent Parser:-

Grammar:

$S \rightarrow AB|BC$

$A \rightarrow a.$

$B \rightarrow b$

$C \rightarrow c.$

i) Procedures:-

SC)

```
{
  if (input == 'a')
  {
    A();
    BC();
  }
  else if (input == 'b')
  {
    B();
    C();
  }
  else
    Error();
}
```

A()

```
{
  match('a');
}
```

BC)

```
{
  match('b');
}
```

C()

```
{
  match('c');
}
```

i) Top-Down parsing:

The parser predicts the correct production using the next input symbol.

→ No backtracking is required.

ii) Predictive matching:

Match (Symbol).

If current symbol matches.

Move to next input.

Else.

Report Error.

iv) Dry Run:-

Input:-

ab.

S

↓

AB

↓

A → a.

↓

B → b.

Accepted

Eg:-

Input:-

ac

S

A → a.

Expected b.

Rejected